

Lab 2 Terraform Documentation

Overview

This Lab 2 stack deploys a private application architecture on Oracle Cloud Infrastructure using Terraform.

It creates:

- 1 Virtual Cloud Network (VCN)
- 1 public subnet for the Application Load Balancer and bastion host
- 1 private subnet for the application instance and File Storage mount target
- 1 private compute instance that serves the application
- 1 OCI File Storage Service file system mounted to the private instance
- 1 public Application Load Balancer that forwards traffic to the private instance
- 1 bastion host to reach the private instance securely
- OCI remote state backend configuration in `backend.tf`

The root module is a thin wrapper around `modules/lab2`, which keeps the stack organized by service.

Architecture

Traffic flow is:

1. Client reaches the public Application Load Balancer on port 80.
2. The load balancer forwards the request to the private instance on port 8080.
3. The private instance serves content from an OCI File Storage mount.
4. The bastion host in the public subnet is used for SSH access to the private instance.

The application files are stored on OCI File Storage Service and are mounted into the private instance during boot.

Repository Structure

Root level files

- [backend.tf](#) defines the OCI remote state backend.
- [main.tf](#) passes all root variables into the `lab2` module.
- [outputs.tf](#) exposes the key outputs from the module.
- [provider.tf](#) declares the OCI provider and sets the region.
- [terraform.tfvars](#) holds the environment-specific values for this lab.
- [variables.tf](#) declares the root module input variables.

Module files

All infrastructure resources live in [modules/lab2](#).

- [modules/lab2/data.tf](#) looks up availability domains.
- [modules/lab2/locals.tf](#) defines naming and bootstrap values.
- [modules/lab2/vcn.tf](#) creates the VCN, gateways, and route tables.
- [modules/lab2/subnet.tf](#) creates security lists and subnets.
- [modules/lab2/storage.tf](#) creates File Storage resources and export configuration.
- [modules/lab2/compute.tf](#) creates the private instance, bastion host, and data lookups for their VNICs.
- [modules/lab2/load_balancer.tf](#) creates the public load balancer, backend set, listener, and backend.
- [modules/lab2/outputs.tf](#) defines module outputs.
- [modules/lab2/variables.tf](#) declares the module input variables.

File-by-File Technical Breakdown

[backend.tf](#)

This file declares the OCI backend block for Terraform state.

Important points:

- The OCI backend stores the state file in Object Storage.
- Backend values cannot use Terraform variables or locals.
- Required backend values are supplied during `terraform init` if the file is left partially configured.

[provider.tf](#)

This file declares the OCI provider version and uses the `region` variable.

[variables.tf](#)

This is the root input contract. It defines all values passed to the module, including:

- tenancy and compartment identifiers
- region
- network CIDRs
- instance image and shape details
- bastion settings
- SSH public key
- load balancer shape

[terraform.tfvars](#)

This file holds the environment values for Lab 2.

Update the placeholders here before applying:

- `compartment_id`
- `tenancy_ocid`
- `instance_image_id`
- `ssh_public_key`

The current values in the file are the ones Terraform will use for this lab unless you override them elsewhere.

[main.tf](#)

This file only calls the module:

- it maps the root variables into `module.lab2`
- it keeps the root stack clean and reusable
- it avoids duplicating service resource definitions at the top level

[outputs.tf](#)

This file exposes the most useful deployment outputs:

- VCN OCID
- private instance OCID and private IP
- bastion OCID and public IP
- load balancer public IPs

[modules/lab2/data.tf](#)

This file resolves the availability domains for the tenancy.

The module uses the first availability domain returned by OCI.

[modules/lab2/locals.tf](#)

This file centralizes reusable values such as:

- resource names
- ports
- mount path
- export path
- cloud-init bootstrap script for the private instance

It is also where the private instance bootstrap script is assembled.

[modules/lab2/vcn.tf](#)

This file creates the networking backbone:

- VCN
- Internet Gateway
- NAT Gateway
- public route table
- private route table

The public route table points to the Internet Gateway.

The private route table points to the NAT Gateway.

[modules/lab2/subnet.tf](#)

This file creates:

- security list for the public subnet
- security list for the private subnet
- public subnet
- private subnet

Security behavior:

- the public subnet allows inbound HTTP to the load balancer
- the public subnet allows SSH to the bastion host from the configured source CIDR
- the private subnet allows application traffic from the public subnet
- the private subnet allows SSH from the public subnet to the private instance

- the private subnet allows NFS traffic internally for File Storage access

modules/lab2/storage.tf

This file creates the OCI File Storage Service components:

- mount target
- export set
- file system
- export

The private instance mounts this export and stores the application files there.

modules/lab2/compute.tf

This file creates two compute resources:

- the private application instance
- the public bastion host

It also reads the VNIC details so Terraform can output the private IP and bastion public IP.

Private instance behavior:

- no public IP
- bootstraps with cloud-init
- mounts File Storage
- writes the application files to the mounted path
- runs a Python HTTP server on port 8080

Bastion host behavior:

- public IP assigned
- placed in the public subnet
- used as the SSH entry point into the private subnet

modules/lab2/load_balancer.tf

This file creates the public Application Load Balancer:

- load balancer
- backend set
- listener on port 80
- backend pointing to the private instance on port 8080

Traffic reaches the app through the load balancer public IP.

[modules/lab2/outputs.tf](#)

This file defines module-level outputs. The root [outputs.tf](#) forwards them to the top level.

Prerequisites

Before running the lab, make sure you have:

- Terraform installed
- OCI CLI configured locally
- OCI API key authentication configured in `~/.oci/config`
- A valid OCI tenancy OCID
- A valid OCI compartment OCID
- A valid Ubuntu image OCID for the target region
- An SSH public key
- Permission to create VCN, compute, load balancer, and File Storage resources
- Permission to use the OCI Object Storage bucket that will hold the Terraform state

Backend and Remote State Setup

The backend is declared in [backend.tf](#), but the actual Object Storage details must be supplied when initializing Terraform.

Typical backend values are:

- namespace
- bucket
- key
- region
- auth mode
- config profile

Example initialization:

```
terraform init
```

Notes:

- The namespace and bucket must already exist in OCI Object Storage.

- Backend settings cannot be read from Terraform variables.
- If the bucket does not exist yet, create it in OCI before running `terraform init`.
- Enable bucket versioning if you want safer state recovery.

How To Run

1. Review and edit the environment values

Check [terraform.tfvars](#) and replace every placeholder value with real OCI values.

At minimum, confirm:

- `compartment_id`
- `tenancy_ocid`
- `instance_image_id`
- `ssh_public_key`
- `bastion_ssh_source_cidr`

2. Format the code

```
terraform fmt -recursive
```

3. Initialize Terraform

Use the OCI backend configuration values for your Object Storage bucket.

```
terraform init
```

4. Validate the configuration

```
terraform validate
```

5. Review the plan

```
terraform plan
```

6. Apply the infrastructure

```
terraform apply
```

How To Check That Everything Works

Terraform-side checks

After apply, run:

```
terraform output
```

You should see:

- `vcn_id`
- `private_instance_id`
- `private_instance_ip`
- `bastion_instance_id`
- `bastion_public_ip`
- `load_balancer_public_ips`

OCI-side checks

In the OCI Console, confirm that these resources exist:

- 1 VCN
- 1 public subnet
- 1 private subnet
- 1 Internet Gateway
- 1 NAT Gateway
- 2 route tables
- 2 security lists
- 1 File Storage mount target
- 1 File System
- 1 Export Set
- 1 Export
- 1 private compute instance
- 1 bastion host

- 1 Application Load Balancer

Application checks

1. Open the load balancer public IP from `terraform output` in a browser.
2. Confirm the page loads and shows the Lab 2 application message.
3. If the page does not load, check the backend health in the OCI load balancer console.

The application should be listening on port 8080 on the private instance, while the load balancer listens on port 80.

Bastion checks

Use the bastion host public IP from `terraform output` :

```
ssh -i ~/.ssh/id_rsa opc@<bastion-public-ip>
```

From the bastion, verify the private instance is reachable:

```
ssh opc@<private-instance-ip>
```

File Storage checks

From the private instance, verify the mount:

```
mount | grep fss
ls -la /mnt/fss
ls -la /mnt/fss/app
```

You should see the application files under `/mnt/fss/app` .

HTTP server checks

From the private instance, verify the app locally:

```
curl http://127.0.0.1:8080
```

From the bastion, verify the private instance responds:

```
curl http://<private-instance-ip>:8080
```

From your workstation, verify the load balancer responds:

```
curl http://<load-balancer-public-ip>
```

Troubleshooting

Availability domain lookup fails

If Terraform reports a null availability domain list, confirm that `tenancy_ocid` is set correctly in [terraform.tfvars](#).

Backend initialization fails

Check that:

- the namespace is correct
- the bucket exists
- the OCI config profile in `~/.oci/config` is valid
- the selected region matches the bucket region

Load balancer is unreachable

Check:

- backend set health in OCI
- security lists on the public subnet
- that the private instance HTTP server is running
- that the backend points to the private instance IP

Private instance cannot mount File Storage

Check:

- the mount target exists
- the private subnet security rules allow NFS traffic
- the mount target IP is reachable from the private subnet
- the private instance cloud-init completed successfully

SSH to the bastion fails

Check:

- `bastion_ssh_source_cidr` matches your real source IP
- the bastion instance has a public IP
- the public subnet security list allows port 22 from your source CIDR

Cleanup

To remove everything created by this lab:

```
terraform destroy
```

If you are using remote state, the state file remains in OCI Object Storage until you delete it separately.